

SPRING BOOT –

Q. what is Spring MVC?

Spring MVC is a web framework inside the Spring Framework used to build **web applications** and **REST APIs** using the **Model-View-Controller** design pattern.

It helps separate:

- **Model** → Business/Data layer
- **View** → UI layer (JSP, Thymeleaf, HTML)
- **Controller** → Handles requests and responses

Q.What is DispatcherServlet?

It is the **front controller** of Spring MVC.

All requests first come to DispatcherServlet.

It:

- Receives requests
- Finds correct controller
- Processes response
- Returns view

Q: What is auto configuration in Spring Boot?

Spring Boot automatically configures beans based on:

- Dependencies present in classpath
- Existing configuration
- Environment properties

Example:

If MySQL + JPA dependencies exist, Spring Boot auto-configures datasource automatically.

Q1. How do we connect to DB in springBoot application?

1. Steps to Connect Database in Spring Boot

Step 1: Add Dependencies

For MySQL + JPA:

Maven Dependencies

```
<dependencies>

    <!-- Spring Boot Starter Data JPA -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- MySQL Driver -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>

</dependencies>
```

2. Configure Database Properties

Inside application.properties

```
# DB URL
spring.datasource.url=jdbc:mysql://localhost:3306/employeedb

# Username
spring.datasource.username=root

# Password
spring.datasource.password=root

# Driver
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Hibernate Properties
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

3. Create Entity Class

```
import jakarta.persistence.*;

@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
```

```
private String name;

private String department;

// Getters and Setters
}
```

4. Create Repository Interface

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {

}
```

Spring Boot automatically creates implementation at runtime.

5. Use Repository in Service

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repo;

    public Employee saveEmployee(Employee emp) {
        return repo.save(emp);
    }

}
```

6. Flow of DB Connection

```
Controller
↓
Service
↓
Repository (JPA)
↓
Hibernate
↓
JDBC
```

↓
Database

Q2. What is DataSource?

`DataSource` is an object used to manage database connections.

Spring Boot automatically creates a `DataSource` bean using `datasource` properties.

Q3. What is Connection Pooling?

Creating DB connections repeatedly is costly.

Spring Boot uses **HikariCP** by default for connection pooling.

Benefits:

- Better performance
- Reuse connections
- Faster DB operations

Q4. What happens internally?

When application starts:

1. Spring Boot reads `datasource` properties
2. Creates `DataSource`
3. Configures Hibernate/JPA
4. Creates DB connections
5. Repository communicates with DB using Hibernate

Q5. How does Spring Boot connect to DB?

Spring Boot uses:

- `DataSource`
- JDBC Driver
- JPA/Hibernate
- Database configuration from `application.properties`

to establish connection automatically.

Q6. Why do we use JPA?

JPA reduces boilerplate JDBC code and provides ORM mapping.

Example:

- Java Object ↔ Database Table

Q7. What is Hibernate?

Hibernate is an ORM framework and default JPA implementation in Spring Boot.

Q8. What is ORM?

ORM (Object Relational Mapping) maps:

- Java Classes → Tables
- Objects → Rows

Q9. What is ddl-auto?

```
spring.jpa.hibernate.ddl-auto=update
```

Controls schema generation.

Options:

- create
- update
- validate
- create-drop

Q6. Difference Between JDBC and JPA?

JDBC	JPA
Manual SQL	Minimal SQL
More boilerplate	Less code
Manual mapping	Automatic ORM

JDBC

JPA

Faster sometimes

Easier development

PHASE WISE NOW – PHASE 1 ->

Q1. What is Spring Boot?

Spring Boot is a framework in the Spring Framework ecosystem that makes it much easier to build Java applications—especially web apps and microservices—quickly and with minimal setup.

In simple terms:

Spring Boot = “Spring, but faster and easier to start with.”

Q2. What is IoC?

IoC (Inversion of Control) is a design principle in which the control of object creation and dependency management is transferred from the programmer to the Spring container. In Spring, the IoC container creates, manages, and injects objects called beans using Dependency Injection.

Q3. What is Dependency Injection (DI)?

In Spring Framework, **Dependency Injection (DI)** is a technique where the Spring container provides the required objects (dependencies) to a class instead of the class creating them manually.

Q4. What are Types of Dependency Injection?

1. **Constructor Injection** - Dependencies are injected through the constructor.

Eg - @Component
 class Car {

 private Engine engine;

```

        @Autowired
        public Car(Engine engine) {
            this.engine = engine;
        }
    }

```

Advantages

- Recommended by Spring
- Ensures mandatory dependencies
- Supports immutable objects
- Easy for unit testing

Disadvantage

- Constructor becomes large if too many dependencies

2. **Setter Injection** - Dependencies are injected using setter methods.

Eg - @Component

```

class Car {

    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}

```

Advantages

- Useful for optional dependencies
- Flexible

Disadvantage

- Object can be partially initialized
- Not immutable

3. **Field Injection** - Dependencies injected directly into fields.

Eg - @Component

```

class Car {

    @Autowired

    private Engine engine;
}

```

}

Advantages

- Less code
- Easy to write

Disadvantages

- Not recommended
- Hard to test
- **Breaks encapsulation** - Field Injection breaks encapsulation because dependencies are injected directly into private fields using reflection, making dependencies hidden and allowing objects to exist in incomplete or invalid states. Constructor Injection preserves encapsulation by enforcing dependency requirements during object creation.
- Cannot create immutable classes

QA. Which DI type is best and why?

Constructor Injection because it promotes loose coupling, immutability, and testability.

QB. When to use Setter Injection?

When dependency is optional.

QC. Why Field Injection is discouraged?

Because it makes unit testing difficult and breaks encapsulation.

Q5. Explain Bean Lifecycle.

In Spring Framework, the **Bean Lifecycle** describes, How a Spring Bean is created, initialized, used, and destroyed by the Spring IoC Container.

Complete Bean Lifecycle Flow -

Container Starts



Bean Instantiation



Dependency Injection



Aware Interfaces



BeanPostProcessor (Before Init)

[postProcessBeforeInitialization()]



Initialization Methods

[@PostConstruct

public void init() {

 System.out.println("Init method");

}]

OR [afterPropertiesSet()]

OR XML – [<bean init-method="init">]



BeanPostProcessor (After Init)

[postProcessAfterInitialization()]



Bean Ready for Use

[Student s = context.getBean(Student.class);]



Container Shutdown - Spring destroys singleton beans.

[context.close();]



Destroy Methods - Cleanup logic executed.

a) @PreDestroy

```
@PreDestroy  
public void cleanup() {  
}
```

b) DisposableBean

```
destroy()
```

c) XML destroy-method

```
<bean destroy-method="cleanup">
```

QA. What is Bean Lifecycle?

Complete process from bean creation to destruction.

QB. Difference between constructor and @PostConstruct?

Constructor

Object creation

@PostConstruct

After dependency injection

QC. Which bean scope supports destroy method?

Singleton scope.

QD. What is BeanPostProcessor?

Interface used to modify beans before and after initialization.

Q6. What is loose coupling?

Loose coupling means reducing dependency between classes so that changes in one class minimally affect another. Spring achieves loose coupling using Dependency Injection and interfaces.

One-Line Memory Trick

Tight Coupling → Direct dependency

Loose Coupling → Dependency via abstraction/interface

Q7. How Spring Achieves Loose Coupling

Spring Framework achieves loose coupling using:

- IoC (Inversion of Control)
- Dependency Injection (DI)

Spring creates and injects dependencies automatically.

Q8. Tight vs Loose Coupling

Tight Coupling	Loose Coupling
Depends on concrete class	Depends on interface
Hard to modify	Easy to modify
Difficult testing	Easy testing
Less flexible	More flexible

QA. Why is loose coupling important?

- Better maintainability
- Easy testing
- Flexibility
- Reusability

QB. How does Spring support loose coupling?

Using IoC Container and Dependency Injection.

Interview Questions

Q1. Why is IoC Needed?

IoC is needed to achieve loose coupling, better maintainability, easier testing, and centralized object management. Instead of classes creating dependencies manually, the Spring container manages and injects them automatically.

Q2. Difference Between Tight Coupling vs Loose Coupling

Tight coupling means classes are heavily dependent on each other, while loose coupling means classes depend on abstractions/interfaces, making the application flexible and maintainable.

Q3. Constructor Injection vs Setter Injection

Constructor Injection injects dependencies during object creation and is preferred for mandatory dependencies. Setter Injection injects dependencies after object creation and is mainly used for optional dependencies.

Q4. Why is Constructor Injection Preferred?

Constructor Injection is preferred because it ensures mandatory dependencies, supports immutability, prevents partially initialized objects, improves encapsulation, and makes unit testing easier.

Q9. What is a Bean in Spring Core?

In Spring Core, a Bean is an object that is instantiated, configured, and managed by the Spring IoC Container. Beans are created using configurations such as `@Component`, `@Bean`, or XML configuration.

QA. Difference between object and bean?

Object	Bean
Created manually using <code>new</code>	Managed by Spring container

QB. Default bean scope?

Singleton

QC. Which annotations create beans?

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`
- `@Bean`

Q10. Bean Scope

Scope	Meaning
Singleton	One object per container (default)
Prototype	New object every request
Request	One bean per HTTP request
Session	One bean per session

Q11. BeanFactory vs ApplicationContext

BeanFactory is the basic Spring IoC container that provides dependency injection and lazy bean loading. ApplicationContext is an advanced container built on top of BeanFactory that provides additional enterprise features like event handling, annotation support, AOP integration, and eager singleton initialization.

QA. Which container is used in Spring Boot?

ApplicationContext

QB. Which one supports lazy loading?

BeanFactory (default behavior)

QC. Which container is preferred in real projects?

ApplicationContext

Q12. What is XML configuration?

XML configuration in Spring is a way of defining beans and dependency injection inside XML files using `<bean>` tags. It was widely used before annotations and Java-based configuration became popular.

XML Configuration	Annotation Configuration
External configuration	Inside Java classes
More verbose	Cleaner

XML Configuration

Harder maintenance
Old approach

Annotation Configuration

Easier maintenance
Modern approach

Q13. Java-Based Configuration in Spring

Java-Based Configuration in Spring is a way of configuring beans using Java classes and annotations like `@Configuration` and `@Bean` instead of XML files. It provides type safety, better readability, and easier maintenance.

Q1. Difference between `@Component` and `@Bean`?

`@Component`

Auto-detected by component scan
Used on class

`@Bean`

Manually declared
Used on method

Q2. Which is preferred: XML or Java Config?

Java-Based Configuration

Q3. Which container loads Java config?

`AnnotationConfigApplicationContext`

Q14. `@Bean` `@Configuration` `@Component` in Spring.

1. `@Component`

`@Component` is used for automatic bean creation through component scanning.

Used For General-purpose bean creation.

Specialized versions:

- `@Service`
- `@Repository`
- `@Controller`

2. `@Bean`

`@Bean` is used for manual bean creation and gives more control over object configuration.

Used When

- Third-party classes

- Custom object creation
- Manual configuration needed

Example with Third-Party Class

```
@Bean
public ObjectMapper objectMapper() {
    return new ObjectMapper();
}
```

Cannot add `@Component` to external library classes.

3. @Configuration

`@Configuration` marks a class as a source of Spring bean definitions and ensures proper bean lifecycle and singleton behavior.

Q15. What are stereotype annotations?

Stereotype annotations in Spring are specialized annotations used to mark classes as Spring-managed components. They include `@Component`, `@Service`, `@Repository`, and `@Controller`, each representing a specific application layer.

Main Stereotype Annotations -

Annotation	Layer	Purpose
<code>@Component</code>	General	Generic Spring bean
<code>@Service</code>	Service layer	Business logic
<code>@Repository</code>	DAO layer	Database operations
<code>@Controller</code>	Web layer	Spring MVC controller
<code>@RestController</code>	REST API layer	REST endpoints

QA. Difference between `@Component` and `@Service`?

Technically: No major difference

But semantically:

1. `@Component` → generic bean 2. `@Service` → business layer bean

QB. Which stereotype handles database exceptions?

`@Repository`

QC. Difference between `@Controller` and `@RestController`?

@Controller **@RestController**

Returns views/pages Returns JSON/XML

Used in MVC Used in REST APIs

Q16. Why use @Repository?

@Repository is used to mark DAO or persistence layer classes in Spring. It helps Spring identify database-related components and provides automatic exception translation by converting database-specific exceptions into Spring's `DataAccessException`.

Example

Without Spring:

```
SQLException  
HibernateException  
PersistenceException
```

Different technologies throw different exceptions.

With @Repository

Spring converts them into:

```
DataAccessException
```

This is called:

```
Persistence Exception Translation
```

Q17. What is Autowiring?

Automatically injecting dependencies (beans) into another bean by the Spring IoC Container.

Example of Autowiring

```
@Component  
class Engine {  
}  
  
@Component  
class Car {  
  
    @Autowired  
    private Engine engine;  
}  
  
o creates Engine  
o injects it into Car Automatically.
```


Problem: Multiple Beans -

Suppose:

```
interface PaymentService {  
}
```

```
@Component  
class UpiPaymentService implements PaymentService {  
}
```

```
@Component  
class CardPaymentService implements PaymentService {  
}
```

Now:

```
@Autowired  
private PaymentService paymentService;
```

Spring gets confused.

Error:

```
NoUniqueBeanDefinitionException
```

Because:

```
2 beans found
```

Solution 1: @Qualifier

Purpose -Used to specify exactly WHICH bean should be injected.

Example

```
@Component("upi")  
class UpiPaymentService implements PaymentService {  
}  
@Component("card")  
class CardPaymentService implements PaymentService {  
}
```

```
@Autowired  
@Qualifier("upi")  
private PaymentService paymentService;
```

Now Spring injects:

```
UpiPaymentService
```

Interview Point

`@Qualifier` is used when multiple beans of same type exist and we want to specify which bean should be injected.

Solution 2: `@Primary`

Purpose

Marks one bean as default/preferred bean.

Example

```
@Component
@Primary
class UpiPaymentService implements PaymentService {
}
```

```
@Component
class CardPaymentService implements PaymentService {
}
```

Now:

```
@Autowired
private PaymentService paymentService;
```

Spring automatically injects:

```
UpiPaymentService
```

because it is marked:

```
@Primary
```

Q18. Can `@Autowired` work without `@Component`?

✗ No

Bean must exist in Spring container.

Q19. Can @Autowired inject interface?

✓ Yes

Spring injects implementing class.

Q20. How does autowiring work internally?

Internally, Spring autowiring works using the IoC container and `AutowiredAnnotationBeanPostProcessor`. During bean creation, Spring scans for `@Autowired`, resolves matching dependencies from the container by type, and injects them using reflection or constructor invocation.

Which class processes @Autowired?

- `AutowiredAnnotationBeanPostProcessor`

Does Spring use reflection for autowiring?

✓ Yes

When does autowiring happen?

During:

Bean initialization phase

How does Spring resolve dependencies?

By:

- type
- qualifier
- primary bean
- bean name

High-Level Flow -

```
Container Starts
    ↓
Bean Definitions Loaded
    ↓
Bean Objects Created
    ↓
Spring Finds Dependencies
    ↓
Dependency Resolution
    ↓
Bean Injected
```

Q21. When is @PostConstruct called?

After dependency injection but before bean usage.

Q22. When is @PreDestroy called?

Before bean destruction during container shutdown.

Q23. Why Spring Boot?

Spring Boot is used to simplify Spring application development by providing auto-configuration, embedded servers, starter dependencies, and production-ready features. It reduces boilerplate configuration and enables faster development of enterprise and microservice applications.

Spring vs Spring Boot

Spring	Spring Boot
Requires heavy configuration	Minimal configuration
External server needed	Embedded server
Manual dependency management	Starter dependencies
Slower setup	Faster development

Q24. Why embedded server important?

No need to deploy WAR file separately.

Q25. What is starter dependency?

Pre-configured dependency bundle.

Q26. What is auto-configuration?

Auto-Configuration in Spring Boot is a feature that automatically configures Spring application components based on dependencies present in the classpath, reducing manual configuration and setup.

Q27. Explain Embedded server.

Embedded Server in Spring Boot means the web server like Tomcat, Jetty, or Undertow is packaged inside the application itself. This eliminates the need for external server installation and allows applications to run directly as executable JAR files.

Q28. Main advantage of embedded server?

Simplified deployment and standalone execution.

Q29. Why executable JAR important?

Application becomes self-contained.

Q30. What is auto-configuration?

Auto-Configuration in Spring Boot is a feature that automatically configures Spring application components based on dependencies present in the classpath, reducing manual configuration and setup.

Real Flow

```
Dependency Added
    ↓
Spring Boot Detects It
    ↓
Related Beans Auto Configured
```

If Spring Boot finds: **spring-webmvc** in classpath, it auto-configures:

- DispatcherServlet
- MVC configuration

If it finds: **spring-data-jpa** it auto-configures:

- DataSource
- Hibernate
- JPA repositories
- TransactionManager

If it finds: **spring-boot-starter-web** it auto-configures:

- Tomcat
- DispatcherServlet
- Jackson
- REST support
- Spring MVC

Q31. Which annotation enables auto-configuration?

`@EnableAutoConfiguration`

Q32. Which annotation internally contains it?

`@SpringBootApplication`

Q33. How does Spring Boot decide what to configure?

By checking:

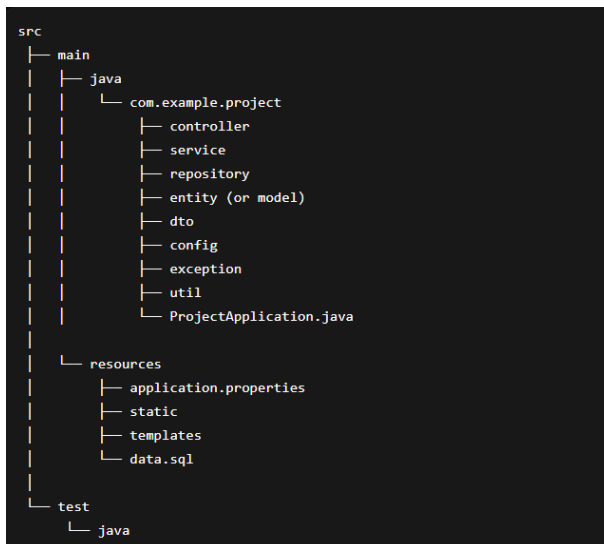
- classpath dependencies
- existing beans
- application properties

Q34. Can we override auto-configured beans?

✓ Yes

Q35. Explain structure of spring boot.

Spring Boot project structure organizes application code into layers like Controller, Service, Repository, Entity, Configuration, and Resources. This layered architecture improves maintainability, scalability, and separation of concerns.



data.sql

Initial database scripts.

Example:

```
INSERT INTO user VALUES(1, 'Sudhanshu');
```

Runs automatically during startup.

Easy Memory Trick -

Controller → API
Service → Logic
Repository → DB
Entity → Table
DTO → Data Transfer
Config → Settings

Q36. Why use layered architecture?

- Loose coupling
- Better maintainability
- Easy testing

Q37. Why use DTO instead of Entity directly?

- Security
- Avoid exposing DB structure
- Better API design

Q38. Which file contains application configuration?

`application.properties`

Q39. What is pom.xml?

`pom.xml` stands for: **Project Object Model (POM)** is the Maven configuration file used to manage project dependencies, build settings, plugins, and project metadata in a Spring Boot application.

Q40. What is dependency management?

Managing library versions automatically.

Q41. Purpose of Maven plugin?

Creates executable Spring Boot JAR.

Q42. Difference between `groupId` and `artifactId`?

groupId	artifactId
Organization	Project name

Q43. What are Starter Dependencies in Spring Boot?

Starter Dependencies in Spring Boot are pre-configured dependency bundles that simplify dependency management by providing all required libraries for a specific functionality in a single dependency.

Q44. Why starter dependencies introduced?

To reduce dependency management complexity.

Q45. What starter is used for REST APIs?

`spring-boot-starter-web`

Q46. Which starter includes Hibernate?

`spring-boot-starter-data-jpa`

Q47. Do starter dependencies improve version compatibility?

✓ Yes Spring Boot manages versions automatically.

Q48. Explain use of main class in spring boot application.

The main class in Spring Boot is the application entry point containing `main()` and `SpringApplication.run()`. It bootstraps the Spring application, creates the IoC container, performs component scanning, auto configuration, and starts the embedded server.

Q49. What is purpose of `SpringApplication.run()`?

Starts Spring Boot application and initializes IoC container.

Q50. Which annotation is mandatory in main class?

`@SpringBootApplication`

Q51. Why keep main class in root package?

To ensure component scanning covers all sub-packages.

Q52. Explain `@SpringBootApplication`.

`@SpringBootApplication` is a convenience annotation in Spring Boot that combines `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`. It enables bean configuration, automatic dependency configuration, and component scanning, simplifying Spring Boot application setup.

Q53. What annotations are present inside `@SpringBootApplication`?

`@SpringBootConfiguration`
`@EnableAutoConfiguration`
`@ComponentScan`

Q54. Can we use all three separately?

✓ Yes

`@Configuration`
`@EnableAutoConfiguration`
`@ComponentScan`

Q55. Which annotation enables auto configuration?

`@EnableAutoConfiguration`

Q56. Which annotation scans beans?

`@ComponentScan`

Q57. Why starter dependencies useful?

Starter dependencies are useful because they reduce dependency management complexity, provide compatible library versions, simplify configuration, enable faster development, and automatically include required transitive dependencies.

Q58. Application Properties in spring boot

`application.properties` in Spring Boot is a configuration file used to externalize application settings such as database configuration, server properties, logging, and custom values. It helps improve maintainability and environment management.

Q59. Default location of `application.properties`?

`src/main/resources`

Q60. Default server port?

8080

Q61. How to inject custom property?

Using: `@Value("${property.name}")`

Q62. Can Spring Boot support YAML?

Yes, Using: `application.yml`

Q63. Common Properties

1 Server Configuration

Change server port: `server.port=9090`

Default:8080

Change application name: `spring.application.name=EmployeeApp`

2 Database Configuration

Example MySQL:

```
spring.datasource.url=jdbc:mysql://localhost:3306/employee_db
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

3 JPA / Hibernate Configuration

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.format_sql=true
```

Property	Purpose
ddl-auto=update	Updates DB schema automatically
show-sql=true	Prints SQL queries
format_sql=true	Formats SQL nicely

4 Logging Configuration

```
logging.level.root=INFO
```

```
logging.level.org.springframework=DEBUG
```

Controls log levels.

Levels:

```
TRACE
DEBUG
INFO
WARN
ERROR
```

5 Custom Properties

You can create your own:

```
app.company=OpenAI
```

```
app.version=1.0
```

Access using:

```
@Value("${app.company}")
private String company;
```

@Value Example

Properties:

student.name=Sudhanshu

Java:

```
@Component
public class Student {

    @Value("${student.name}")
    private String name;

}
```

Spring injects: Sudhanshu automatically.

Environment-Specific Properties

Development:

application-dev.properties

Production:

application-prod.properties

Activate profile:

spring.profiles.active=dev

Spring loads:

application-dev.properties

Q64. application.properties vs application.yml

Properties:

```
server.port=8080

spring.datasource.username=root
```

YAML:

```
server:
  port: 8080

spring:
  datasource:
    username: root [Both work. YAML is more readable for nested config.]
```

Q65. Difference between properties and YAML?

`application.properties` uses key-value format while YAML uses hierarchical indentation-based format. YAML is more readable and better for nested configuration, whereas properties files are simpler and easier for small applications.

Q66. How profiles work in spring boot?

Spring Profiles in Spring Boot allow environment-specific configuration by loading profile-specific property files like `application-dev.properties` or `application-prod.properties`. Active profiles are configured using `spring.profiles.active`

Examples -

application.properties

```
spring.profiles.active=dev
```

This activates:

```
application-dev.properties
```

application-dev.properties

```
server.port=8081
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/dev_db
```

application-prod.properties

```
server.port=9090
```

```
spring.datasource.url=jdbc:mysql://prod-server:3306/prod_db
```

Example Override

application.properties

```
server.port=8080
```

application-prod.properties

```
server.port=9090
```

Active profile:

```
spring.profiles.active=prod
```

Result: 9090

Using @Profile

You can activate beans only for specific profiles.

Example:

```
@Configuration
@Profile("dev")
public class DevConfig {

}
```

Bean created only when:

```
spring.profiles.active=dev
```

Common Interview Questions-

Q1. Why profiles needed?

Environment-specific configuration.

Q2. Which property activates profile?

```
spring.profiles.active=prod
```

Q3. Which configuration gets higher priority?

Profile-specific properties.

Q4. Can multiple profiles be active?

✓ Yes `spring.profiles.active=dev,test`

QQ67.